Predicting flow with Back Propagation Neural Networks

F328799

Contents

Chapter 0

Outline

The aim of this project is to predict the flow-rate of the River Ouse at Skelton in advance to predict the flood potential. We will use a Back Propagation Neural Network to predict the flow-rate of the river, using data from the surrounding catchment area. This data includes river flow-rate from Skip Bridge, Crakehill and Westwick, as well as rainfall data from East Cowton, Arkengarthdale, Snaizeholme and Malham Tarn. The data was collected between 01/01/1993 to 31/12/1996.

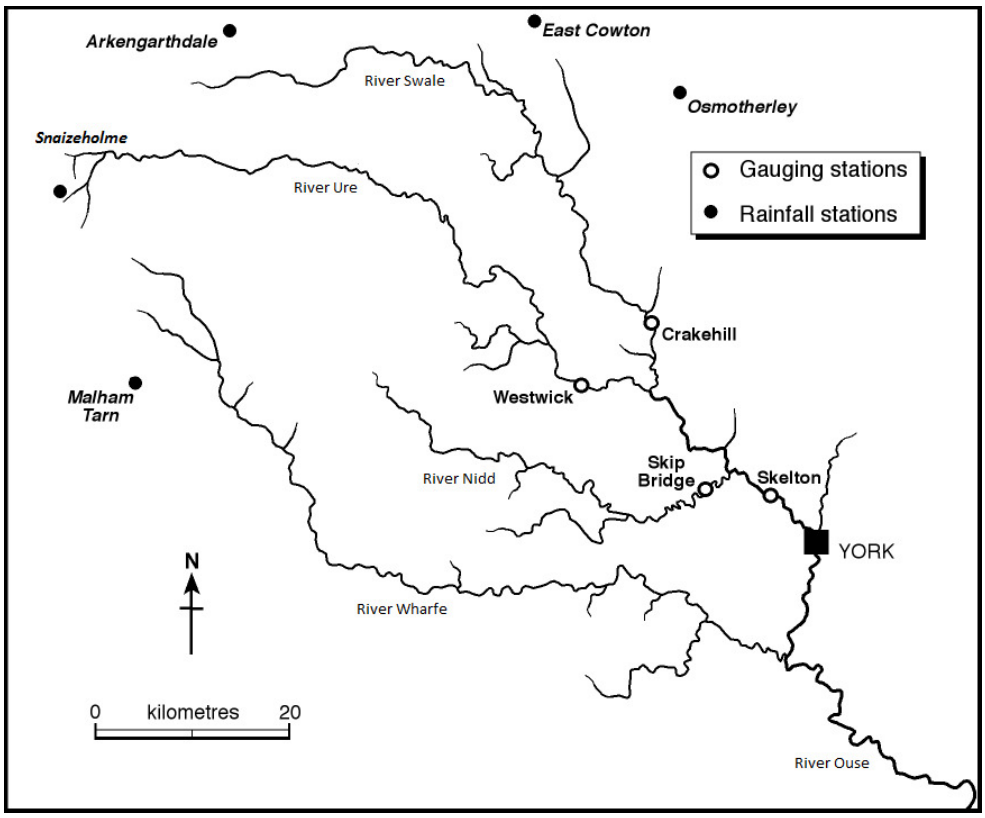


Figure 0.0.1: River Ouse Catchment

The sections of this report will include:

1. Data Pre-processing
2. Implementing the Neural Network
3. Model Training
4. Evaluation

0.1 System Environments

The ANN will be programmed using python 3.13.2 with a conda virtual environment. The libraries used will include:

- numpy
- pandas, sqlalchemy

- openpyxl
- sqlite3
- matplotlib

The data will be stored in an SQLite database, this is for ease of access and to keep the data in a single location, with concise SQL commands to access the data.
The data will be split into training and testing data, with 60% of the data used for training, 20% used for testing and 20% for evaluation.

Chapter 1

Data Pre-processing

1.1 Importing data

The first step is to import the data from the give xlsx file into the database. This will be done using the pandas library to read the data from the xlsx file and then using the sqlalchemy library to write the data to the database. For simplicity, I created a second xlsx file without the first row of headers, this is so that the data can be read into the database without any issues.

This code shows how I handled the non-numerical data in the xlsx file. It is then imported into the database using the sqlalchemy library. Line 16 is used to remove non-numerical data when importing into the database. This is to avoid any type-errors when the create_engine creates the dataBase.

```
1  excel_import.py
2
3      import pandas as pd
4      import numpy as np
5      import sqlite3
6      from sqlalchemy import create_engine
7
8
9      def data_import(dataBase:pd.DataFrame, dataTable:str, excelFile:str):
10         '''Function for importing Excel data into a database'''
11
12         # Read the excel file and store the data in a DataFrame
13         excelFile = pd.read_excel(excelFile)
14
15         # Check for non-numeric values in non-first columns and replace them with na points
16         excelFile.iloc[:, 1:] = excelFile.iloc[:, 1:].apply(pd.to_numeric, errors='coerce').fillna(np.nan)
17
18         # Create a connection to the SQLite database (or create it if it doesn't exist)
19         conn = sqlite3.connect(dataBase)
20         cursor = conn.cursor()
21
22         # Drop the table if it exists so the database can be recreated from scratch for testing
23         cursor.execute(f'DROP TABLE IF EXISTS {dataTable}')
24
25         # Use SQLAlchemy to import the DataFrame into the SQL database
26         engine = create_engine(f'sqlite:///{dataBase}')
27         excelFile.to_sql(dataTable, engine, if_exists='replace', index=False)
28
29         # Commit the changes and close the connection
30         conn.commit()
31         conn.close()
32         print('Data imported successfully!')
```

All code will be enclosed in the additional files included

1.2 Extreme Outliers

Using the database as a source of data, we can now begin to pre-process the data. Using matplotlib, we can plot the data to see if there are any trends or patterns in the data, along with any outliers that may need to be removed.

In the given dataset, there were a number of values that were not possible, such as negative flow rates and rainfall values in the thousands. These values need to be removed from the dataset to ensure the model is accurate. To remove these and other 'extreme' outliers, I used the inter-quartile range (IQR) method. When I implemented this, I used a variable to multiply by the IQR to get the upper and lower bounds, allowing me to refine the data to a point I thought was acceptable.

A typical weight for extreme values is 3, this is because the IQR is the range between the 25th and 75th percentile, so multiplying by 3 gives a range of 99.7% of the data (when its normally distributed). However, as the data is skewed heavily towards 0, the 75th percentile still encapsulates a large amount of possible data within a wide range. After experimenting with different values, the results were as follows:

Weight	Amount Removed
1.5	2031
3.0	782
6.0	176
10.0	50

Table 1.1: Results of different weights for IQR

From testing, I decided that 10 was the best weight to use, as it removed the most extreme values without removing too much data. The remaining data looks real and is within a reasonable range for realistic rainfall and flow rates.

1.3 Averaging and Deviations

The data has had its extreme outliers removed, but it is still not in a usable state. The data needs to be averaged and standardised to ensure that the data is in a usable state for the neural network.

> *For context: These functions are called within an iterative loop that acts on them for each of the columns in the database. columnData is a pandas data frame of the date with that dates record.*
>
> *The data points are uniformly iterated with columnData.itertuples(), hence the use of the point[n] convention to access the data.*
>
> *When a data point is removed, it is stored in a dictionary object, with the key being the column name and the value being a dictionary of the removed data points. All code will be enclosed in the additional files included.*

Our IQR with extreme weighting method does not account for large amount of rainfall in a dry spell, or low flow rate at a time of flooding. To account for this, we will use methods to determine if the data is within a reasonable range, over a period of time.

Rainfall and especially flow rate data move in trends, so to best capture the data we will use a condensed version of the data, with only its neighbors. We will calculate our outliers over this as rainfall in summer will not be as high as in winter (it's still England though), so calculating the averages of the whole data set will not be accurate.

data_cleaning.py

```
import numpy as np
import pandas as pd

def local_range_df(columnData:pd.DataFrame, dataPoint:tuple, columnLength:int, windowSize:int) -> pd.DataFrame:
    '''Function to create a smaller range of data around a point'''
    windowSplit = round(windowSize/2)

    # Defines the start and end of the range
    if int(dataPoint[0])-windowSplit < 0:
        '''If the point is close to 0'''
        start = 0
        end = windowSize-int(dataPoint[0])
    elif int(dataPoint[0])+windowSplit > columnLength:
        '''If the point is close to the end of the DF'''
        end = columnLength
        start = columnLength - windowSize
    else:
        '''If the point is in the middle of the DF'''
        start = int(dataPoint[0])-windowSplit
        end = int(dataPoint[0])+windowSplit

    # Create a range of the data around the point
    localData = columnData.loc[start:end]

    return localData
```

We will pass this local data to the standard deviation and IQR functions to calculate the upper and lower bounds of the data.

Flow rate of a river rises and falls in patterns, so to best catch outliers we will use a mean and standard deviation over a period of time.

data_cleaning.py

```
    import numpy as np
    import pandas as pd

    def standard_deviation_calculator(columnData:pd.DataFrame, columnName:str, deviationWeight:int) -> tuple[float, float]:
        '''Function to calculate the standard deviation of a range of data'''

        # Calculate the mean and standard deviation of the DataFrame
        deviation = columnData[columnName].std(axis=0, skipna=True, numeric_only=True, ddof=1)
        average = columnData[columnName].mean(axis=0, skipna=True, numeric_only=True)

        # Calculate the upper and lower bounds
        lowerBound = average - (deviationWeight * deviation)
        upperBound = average + (deviationWeight * deviation)

        return lowerBound, upperBound
```

This method determines if the data follows the trend of its neighbors, and if it does not, it is removed from the dataset. One day of high flow rate isn't a realistic measurement as the river constantly flows.

However, rainfall can be more sporadic, so we will once again use a weighted IQR method to remove extreme values.

data_cleaning.py

```
    import numpy as np
    import pandas as pd

    def iqr_calculator(columnData:pd.DataFrame, columnName:str, deviationWeight:int) -> tuple[float, float]:
        '''Function to calculate the IQR of a range of data'''

        # Calculate the IQR of the range
        Q1 = columnData[columnName].quantile(0.25)
        Q3 = columnData[columnName].quantile(0.75)
        IQR = Q3 - Q1

        # Calculate the upper and lower bounds, round to 3 decimal places
        lowerBound = round(Q1 - (deviationWeight * IQR), 3)
        upperBound = round(Q3 + (deviationWeight * IQR), 3)

        return lowerBound, upperBound
```

This will be done over a period of time, as a single day of high rainfall is possible in a storm, before a low rainfall spell occurs again. By splitting the data this way we allow the more accurate flow rate data to be used in the model, while still allowing for the sporadic nature of rainfall.

Once the data points have been removed from the dataset, we can replace them with the average of the surrounding data points. This is to ensure that the data is continuous and that the model can be trained on the data.

1.4 Interpolation

When the algorithm removes the data points, it leaves gaps in the data. I made this decision to separate the extraction and interpolation for two reasons:

1. To allow for efficiency of interpolation
2. For accuracy, as im only using the data points that were recorded, not interpolated

To interpolate the data, I used a simple moving average (SMA) method to fill the gaps. I implemented this with a variable window size for the SMA to allow for flexibility in the data. The numpy array allows for skipping of NaN values, so the data can be interpolated in places where I have removed the data points. However, too small of a window size ends up with a variation of a forward fill at points with data removed. I decided that a window size of 6 was the best for the data, allowing for trends to show without ending up with a forward fill.

data_cleaning.py

```
    import numpy as np
    import pandas as pd

    def simple_moving_average(columnData:pd.DataFrame, columnName:str, windowSize:int, removedDict:dict, updatedDict:dict) -> tuple[dict, dict]:
        '''Function to update empty points in the dataset'''

```

```
        # Find all NaN values in the column
        emptyPoints = columnData[columnData[columnName].isna()]

        # Iterate through the empty points
        for point in emptyPoints.itertuples():
            '''Iterate through the empty points in the DataFrame'''

            # Get the local range of data around the point
            localData = local_range_df(columnData, point, len(columnData), windowSize)

            # Calculate the average of the range
            average = round(localData[columnName].mean(skipna=True),)

            # Update the statistics
            removedDict[columnName][point[1]].append(float(average))

            # For visualisation purposes, store some of the updated data in separate dictionary
            try:
                updatedDict[columnName][point[1]].append(float(average))
            except:
                None

            # Replace the empty point with the average
            columnData.loc[point[0], columnName] = average

        return removedDict, updatedDict
```

The outputs of this function are two dictionary objects. As explained in the context note, the dictionary is used to store the data points that were removed from the dataset. Both of them are the same, except updatedDict is used for visualisation purposes. As the extreme values are present in the removedDict, when plotted they skew the graph so you can't see the trends in the data.

I then go on to use the updatedDict to plot the data, showing the trends in the data, along with the removed data and the ones that replaced it. This is to show the accuracy of the interpolation and to show the trends in the data. The removedDict is used to update the SQL database, so the data can be used in the neural network; we need a record of data that was changed to be able to update it. If we instead saved it to the data frame, we would lose the location of updated data points. This would result in updating the whole database with the whole data frame, wasting a lot of time and resources replacing points with values they already have. The database has 11,688 data points, with 992 (for variables stated in figure 1.5.2) removed and replaced. That's 8.5% of the data and more than 10 times more efficient only updating those.

1.5 Plotting trends

Once the data has been cleaned and interpolated, we can plot the data to see the trends in the data. We can also make judgments on the variables that determine which point are outliers and which are not.

Lets use the data at Crakehill as an example location for visualisation:

Figure 1.5.1 show us the data at Crakehill, with the unrealistic data present. As I stated before, the data scale is affected by the outliers, so the trends in the data are not visible.

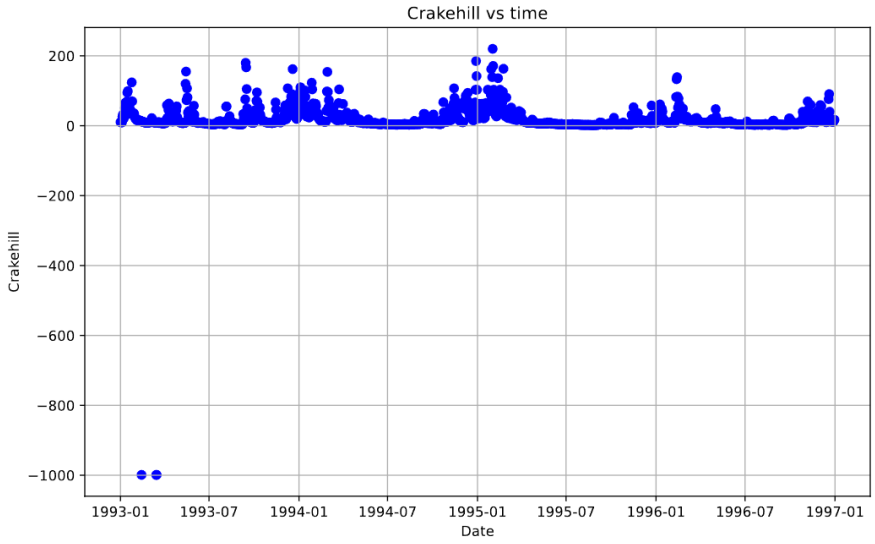


Figure 1.5.1: Crakehill Data with unrealistic data

Figure 1.5.2 shows the data at Crakehill, with the unrealistic data points removed, but still containing outliers

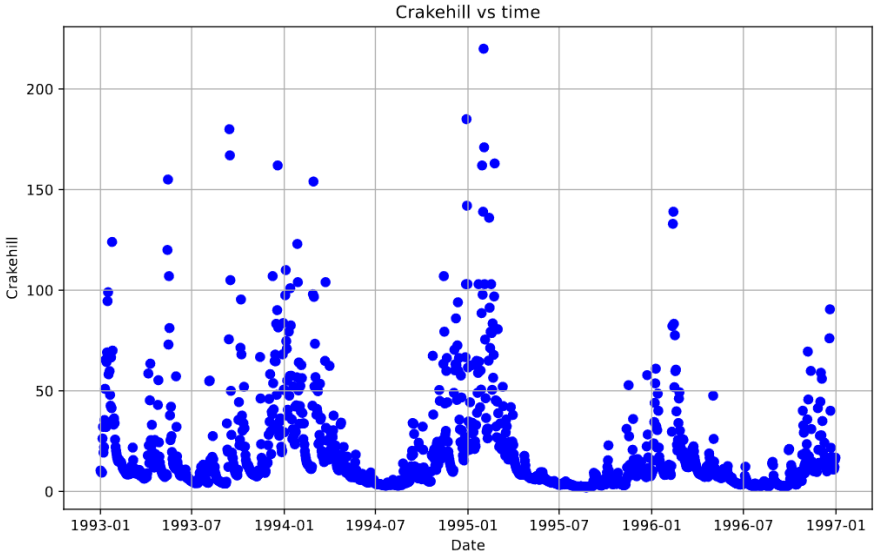


Figure 1.5.2: Crakehill Data without unrealistic data

Finally, figure 1.5.3 shows us the result of our interpolation, with the data points that were removed and replaced. Blue points are the data we are using, red points are the data that was removed and green being what replaced them.

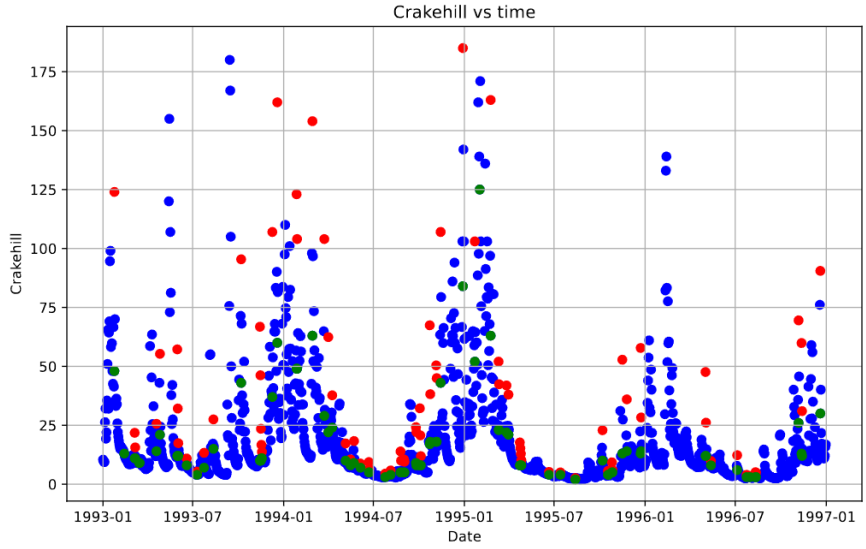


Figure 1.5.3: Crakehill Data interpolated

This specific figure has:

- IQR weight of 1.5
- Standard deviation weight of 2
- Window size of 10 for removal
- Window size of 6 for interpolation
- 91 data points removed and replaced (992 across all locations)

When looking at figure 1.5.2, we can see that in 1993, the flow rate data was much more sporadic than the following years, then in figure 1.5.3, the data is clearly less sporadic and follows the trends more closely. This data should now be considered usable to train our Multi-Layer Perceptron (MLP) model. The database can now be updated with the new data.

1.6 Correlation

To best predict our data, we need to choose the correct inputs that will give us the best results. We need to aim to a high correlation of the data points to the predictand, while also avoiding correlation between predictors.

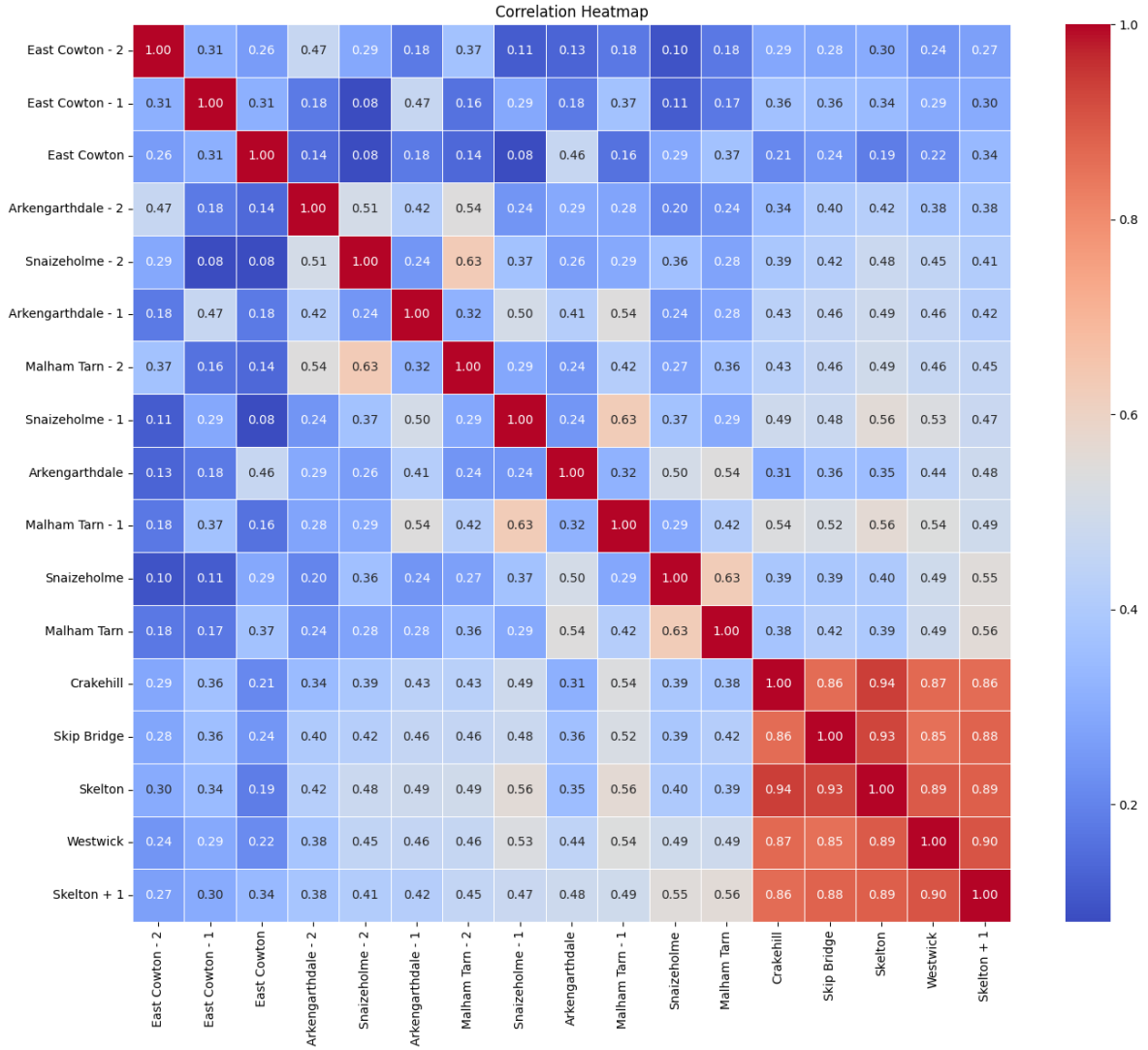


Figure 1.6.1: Correlation between the data points

FIgure 1.6.1 shows the correlation between the data points, which we can use to determine the best predictors for our model. As we are trying to to predict the flow rate at Skelton for the day after a data point, the correlation map has been ordered by the correlation to Skelton + 1. The highest correlation points are the other flow rate data points, with the highest being the flow rate at Westwick. They are all highly correlated to Skelton +1, but also to each other. This is to be expected as they are all flow rate data points of the same river. An interesting point to note is that Westwick is the least correlated to Skelton on the day of measurement, but most correlated to the day after.

The rainfall data is much less correlated to the flow rate data, with the highest correlation being 0.56 at Malham Tarn. This is to be expected as the rainfall data is not directly related to the flow rate data, but the flow rate data is related to the rainfall data. The next highest correlated rainfall location is Snaizeholme (0.55), but this also has a high correlation to Malham Tarn. Refering back to figure 0.0.1, we can see that they are very close, and therefore the data will be very similar.
The next best correlations are at Malham Tarn -1 (0.49) and Arkengarthdale(0.48), but these are are not nearly as correlated each other and the previous two.

My initial choice for predictors will be Westwick, Snaizeholme and Malham Tarn-1, as they are the most correlated to Skelton +1, but also the least correlated to each other. Another option for predictors would be any of the other flow rate data points, and a combination of the 4 mentioned rainfall data points.

We can also use some more complex predictors to see if they are more correlated to the predictand. These complex predictors will be using averages of the data points, and adding select ones together to get a higher correlation.

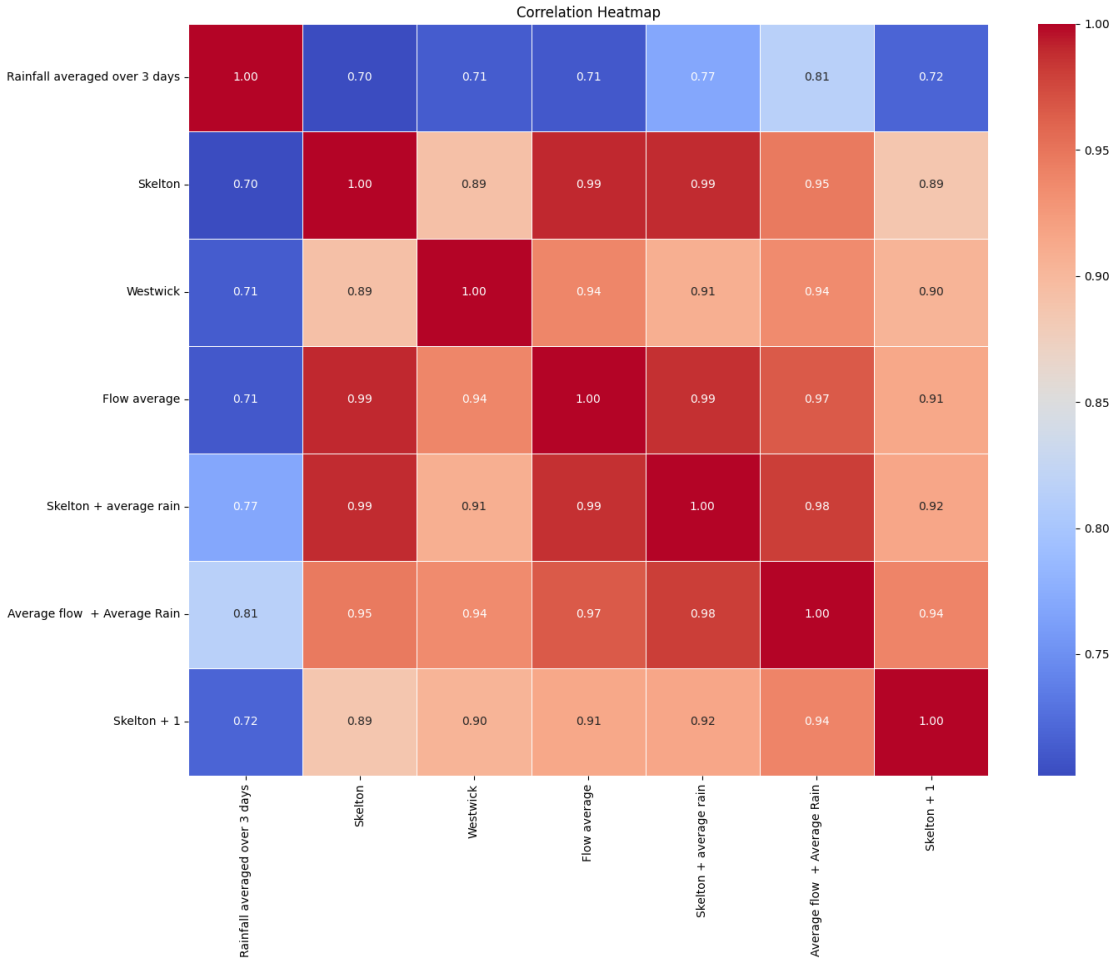


Figure 1.6.2: Complex Correlation between the data points

Figure 1.6.2 shows some of the selected complex data points. In a network of this size, the correlation between points is not as important as the correlation to the predictand, however I don't want to derive complex points that are too mathematically similar to one another.

correlation_mapping.py

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import reading_and_writing as rw

# Read the data
dataFrame = rw.read_data_all(dataBase, dataTable).iloc[:, 1:]

# Formulate the complex predictors
correlationTable['Skelton + 1'] = dataFrame['Skelton'].shift(-1)
correlationTable['Average flow  + Average Rain'] = (dataFrame['Flow average'] + dataFrame['Rainfall average'])
correlationTable['Skelton + average rain'] = (dataFrame['Skelton'] + dataFrame['Rainfall average'])
correlationTable['Flow average'] = (dataFrame['Skelton'] + dataFrame['Westwick'] + dataFrame['Skip Bridge'] + dataFrame['Crakehill']) / 4
correlationTable['Westwick'] = (dataFrame['Westwick'])
correlationTable['Skelton'] = dataFrame['Skelton']
correlationTable['Rainfall averaged over 3 days'] = dataFrame['Rainfall average'] + dataFrame['Rainfall average -1'] + dataFrame['Rainfall average -2']

# Drop na values
correlationTable = correlationTable.dropna()

# Calculate the correlation matrix
correlationMatrix = correlationTable.corr()

# Sort the correlation matrix
correlationMatrix = correlationMatrix.sort_values(by='Skelton + 1', axis=0, ascending=True)
correlationMatrix = correlationMatrix.sort_values(by='Skelton + 1', axis=1, ascending=True)


# Plot the correlation matrix
```

```
plt.figure(figsize=(16, 12))
sns.heatmap(correlationMatrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

Chapter 2

Implementing the Neural Network

The neural network will be implemented using python and trained with the back propagation algorithm. The network will be a multi-layer perceptron (MLP), with at least one hidden layer.

Back propagation works by calculating the expected output of a network, then comparing it to the actual output; this is known as the forward pass. The algorithm then works backwards through the network, adjusting the weights of the network to reduce the error between the expected and actual output - the backward pass. This process is repeated for a given amount of epochs to help reduce the error to a minimal amount. This section is about the implementation of the algorithm and its improvements.

2.1 Data Splitting

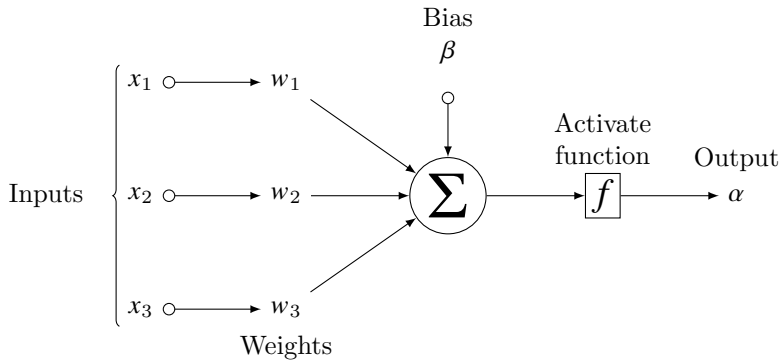
The data will be split into training, testing and evaluation data. To avoid seasonality in the data, we can use 2 years out of the 4 years for training, and the remaining 2 years for testing and evaluation(1 each). On paper this seems like a good idea, as each section has the same seasons for the same amount of time, in the same positions.

However, the data across the years is not equal, with 1993 having much more sporadic flow data than the following years. Additionally, there is the largest spike in flow data in the winter of 1994. This creates an issue where our training may not have sufficient data to predict the spike in flow rate into January 1995. We can try a random split of the data to see if this improves the model, but the results will only be known after the model has been trained.

2.2 Model Architecture

The MLP Architecture consists of repeated layers of nodes, each representing a 'neuron' in the network. Each node takes in x amounts of inputs, and eventually outputs a single value α.
To achieve this, each input is multiplied by a weight, and then summed together with a bias term. This bias term is introduced to avoid the network being stuck at 0, as the weights are randomly initialised, and input data can be 0. Once this sum has been calculated, it is passed through an activation function to give the output of the node.

This output then becomes the input for the next layer of nodes, repeating the process until the output layer is reached.



An example node in a neural network

2.3 Activation Functions

2.4 Cost Functions

There are 2 main loss functions that can be used in a neural network, the mean squared error (MSE) and the mean absolute error (MAE). The MSE is the most common loss function used in neural networks, as it is differentiable and

convex. The MAE is less common, but is more robust to outliers in the data.

We use the cost function to calculated the error of the network, and then use the gradient of the cost function to adjust the weights of the network.

2.4.1 Mean Squared Error

The MSE equation is as follows:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(E_m - O_m)^2 \tag{2.4.1}$$

Where E_m is the expected output and O_m is the actual output.

We differentiate this with respect to output O_m to get the gradient of the cost function.

$$\frac{\partial \text{MSE}}{\partial O_m} = \frac{-2}{n}(E_m - O_m) \tag{2.4.2}$$

For large datasets, this can make the error of the network small and meaningless, leading to large training times.

2.4.2 Mean Absolute Error

The MAE equation is as follows:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|E_m - O_m| \tag{2.4.3}$$

We differentiate this with respect to output O_m to get the gradient of the cost function.

$$\frac{\partial \text{MAE}}{\partial O_m} = \frac{1}{n}\sum_{i=1}^{n}\frac{E_m - O_m}{|E_m - O_m|} \tag{2.4.4}$$

This gives us the following results for each case:

$$\frac{\partial MAE}{\partial O_m} = \begin{cases} \frac{1}{n}, & \text{if } O_m > E_m \\ -\frac{1}{n}, & \text{if } O_m < E_m \end{cases} \tag{2.4.5}$$

As you cannot divide by 0, there is not a smooth gradient at $O_m = E_m$.

Another potential issue is that as the gradient does not rely on the output, the network may not learn as well as with the MSE. This however is its strength, as it is more robust to outliers in the data, as it won't skew the data as much as the MSE.

For this project, I will use the MSE as the loss function, as the data has been cleaned and interpolated, and the outliers have been removed. However, I will use a variation of the MSE.

The function I will use is:

$$\frac{\delta MSE}{\delta O_m} = C(E_m, O_m) = E_m - O_m \tag{2.4.6}$$

This is because the gradient is dependent on the output, and the network will learn better with this function. It also removes the negative gradient as a negative gradient pushes the network in the wrong direction with my implementation of the back propagation algorithm. I have also removed the division by n as it will stifle my learning rate, as the data is already normalised and n is a constant.

2.5 Forward Pass

For n inputs, m hidden nodes in layer $\mathcal{L}$ and q output nodes.

The input matrix takes the n data points from the training data in the form:

$$\text{Matrix } \mathbf{I} \rightarrow [1 \times n] : \mathbf{I} = \begin{vmatrix} \mathbf{I}_1, & \mathbf{I}_2, & \cdots, & \mathbf{I}_n \end{vmatrix} \tag{2.5.1}$$

The weight matrix ω^1 is a $n \times m$ matrix, with each row representing the weights of a input nodes to each connected hidden node in the first hidden layer.

$$\text{Matrix } \omega^1 \rightarrow [n \times m] : \omega^1 = \begin{vmatrix} \omega^1_{11}, & \omega^1_{12}, & \cdots, & \omega^1_{1m} \\ \omega^1_{21}, & \ddots & & \omega^1_{2m} \\ \vdots & & \ddots & \vdots \\ \omega^1_{n1}, & \omega^1_{n2}, & \cdots, & \omega^1_{nm} \end{vmatrix} \tag{2.5.2}$$

The matrix multiplication of the input matrix and the weight matrix gives us a matrix of dimensions $1 \times m$. This correlates to the amount of nodes in the first hidden layer, a sum of weights multiplied by their corresponding input data for each hidden node.

$$\text{Matrix } \mathbf{I} \cdot \omega^1 \rightarrow [1 \times n] \cdot [n \times m] \rightarrow [1 \times m] : \mathbf{I} \cdot \omega = \left| \mathbf{I}_1 \cdot \omega^1_{11} + \cdots + \mathbf{I}_n \cdot \omega^1_{n1}, \quad \cdots, \mathbf{I}_m \cdot \omega^1_{1m} + \cdots + \mathbf{I}_n \cdot \omega^1_{nm} \right| \tag{2.5.3}$$

Each node in the hidden layer will have a bias term β. Formatting this as a matrix, we get a $1 \times m$ matrix, which aligns with the dimensions matrix $\mathbf{I} \cdot \omega$ (2.5.3).

$$\text{Matrix } \beta^1 \rightarrow [1 \times m] : \beta^1 = \left| \beta^1_1, \quad \beta^1_2, \quad \cdots, \quad \beta^1_m \right| \tag{2.5.4}$$

We apply an element-wise summation of the matrix $\mathbf{I} \cdot \omega$ and the bias matrix β, to find the matrix $\mathcal{S}^1 \rightarrow [1 \times m]$: the summation of each node in layer 1. Applying the activation function $\mathcal{F}_1$ to the summation matrix $\mathcal{S}$ gives us the output α of the hidden layer.

Note that the activation function is labeled $\mathcal{F}_1$ as it is the first activation function in the network, and can differ from other activation functions in the network.

$$\text{Matrix } \alpha \rightarrow [1 \times m] : \alpha = \mathcal{F}_1(\mathcal{S}^1) = \mathcal{F}_1(\mathbf{I} \cdot \omega^1 + \beta^1) \tag{2.5.5}$$

$$\text{where } \mathcal{S}^1_m = \sum_{k=1}^{n} \left(\mathbf{I}_k \cdot \omega^1_{km} \right) + \beta^1_m \tag{2.5.6}$$

This activation function is applied to every element in the matrix. This matrix α is then used as the input for the next hidden layer of the network, repeating the same process as before all the way to the output layer:
The weight matrix $\omega^{\mathcal{L}}$ is a $n \times m$ matrix, with each row representing the weights of a hidden node (n) in layer $\mathcal{L} - 1$ to each connected node(m) in layer $\mathcal{L}$.

$$\text{Matrix } \omega^{\mathcal{L}} \rightarrow [n \times m] : \omega^{\mathcal{L}} = \begin{vmatrix} \omega^{\mathcal{L}}_{11}, & \omega^{\mathcal{L}}_{12}, & \cdots, & \omega^{\mathcal{L}}_{1m} \\ \omega^{\mathcal{L}}_{21}, & \ddots & & \omega^{\mathcal{L}}_{2m} \\ \vdots & & \ddots & \vdots \\ \omega^{\mathcal{L}}_{n1}, & \omega^{\mathcal{L}}_{n2}, & \cdots, & \omega^{\mathcal{L}}_{nm} \end{vmatrix} \tag{2.5.7}$$

(2.5.8)

$$\text{Matrix } \beta^{\mathcal{L}} \rightarrow [1 \times m] : \beta^{\mathcal{L}} = \left| \beta^{\mathcal{L}}_1 \ , \quad \beta^{\mathcal{L}}_2 \ , \quad \cdots, \quad \beta^{\mathcal{L}}_m \ \right| \tag{2.5.9}$$

for l hidden layers.

The same summation applies to the hidden layer output α and the weight matrix ω for each hidden layer, all the way to the output layer. We travel layer by layer to reach our predicted output, O_q.

$$\text{Matrix } \alpha^{\mathcal{L}} \rightarrow [1 \times m] : \alpha^{\mathcal{L}} = \mathcal{F}_{\mathcal{L}}(\mathcal{S}^{\mathcal{L}}) = \mathcal{F}_{\mathcal{L}}\left(\alpha^{\mathcal{L}-1} \cdot \omega^{\mathcal{L}} + \beta^{\mathcal{L}} \right) \tag{2.5.10}$$

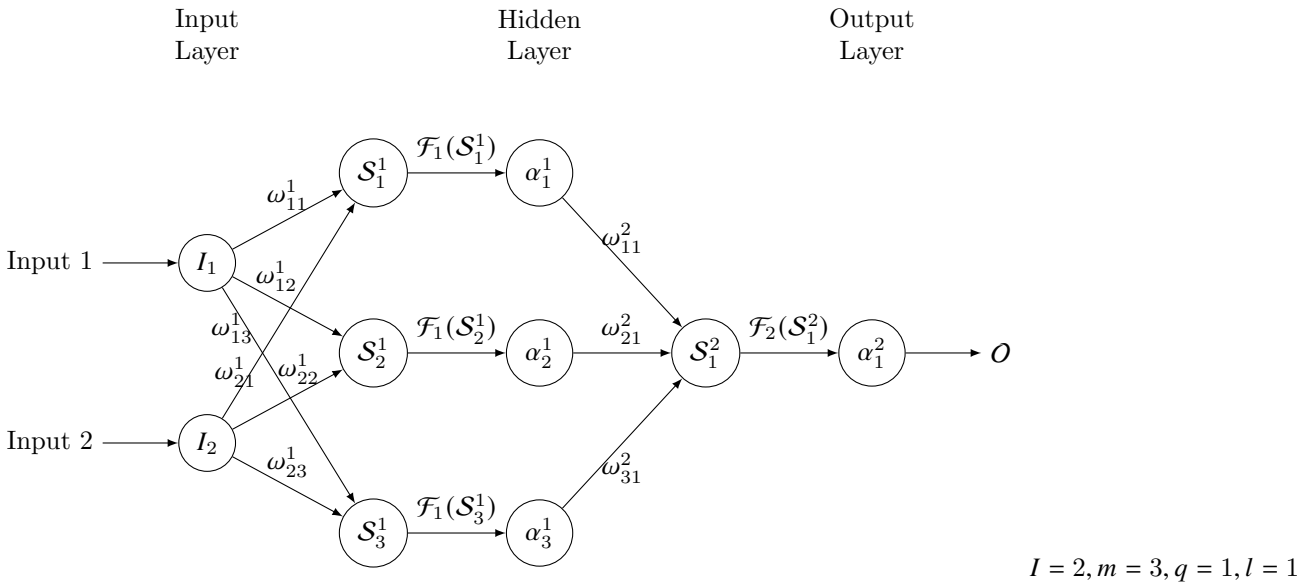
$$\text{where } \mathcal{S}^{\mathcal{L}}_m = \sum_{k=1}^{n} \left(\alpha^{\mathcal{L}-1}_k \cdot \omega^{\mathcal{L}}_{km} \right) + \beta^{\mathcal{L}}_m \tag{2.5.11}$$

Note that $\mathcal{F}_{\mathcal{L}}$ is the activation function at layer $\mathcal{L}$.
Finally, we calculate the output of the network, O_q.

$$\text{Output } \rightarrow [1 \times q] : O_q = \mathcal{F}_{\mathcal{L}}(\mathcal{S}^{\mathcal{L}}) = \mathcal{F}_{\mathcal{L}}(\alpha^{\mathcal{L}-1} \cdot \omega^{\mathcal{L}} + \beta^{\mathcal{L}}) \tag{2.5.12}$$

This gives us our prediction from the forward pass through the network.

An example of a network with 1 hidden node is given below:



$I = 2, m = 3, q = 1, l = 1$

2.6 Backward Pass

The backward pass is used to update the weights and biases of the network, to reduce the error of the network and improve the accuracy of the model. Back propagation is about finding how much of the error is due to each weight in the network, and then updating the weights to reduce the error.

Leibniz's notation of the chain rule is used to find the partial derivative of the error with respect to the weights and biases in the network. Leibniz's notation states that *If a variable z depends on the variable y, which itself depends on the variable x (that is, y and z are dependent variables), then z depends on x as well, via the intermediate variable y.*

$$\frac{\delta z}{\delta x} = \frac{\delta z}{\delta y} \cdot \frac{\delta y}{\delta x} \tag{2.6.1}$$

To back propagate through the network, we need to find how much of the error (cost C) is due to each node in the network so that we can update the weights and biases to reduce the cost for future predictions. Using the chain rule, we can find the rate of change of the error with respect to the weights and biases in the network, working backwards from the output to each node. We can use partial derivatives to find the rate of change of the error with respect to each weight and bias in the network, by differentiating the weights and biases dependent variables.

$$\frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \frac{\delta C}{\delta \mathcal{S}_m^{\mathcal{L}}} \cdot \frac{\delta \mathcal{S}_m^{\mathcal{L}}}{\delta \omega_{nm}^{\mathcal{L}}} \qquad \frac{\delta C}{\delta \beta_{nm}^{\mathcal{L}}} = \frac{\delta C}{\delta \mathcal{S}_m^{\mathcal{L}}} \cdot \frac{\delta \mathcal{S}_m^{\mathcal{L}}}{\delta \beta_{nm}^{\mathcal{L}}} \tag{2.6.2}$$

Using the equation for $\mathcal{S}$ that we found in (2.5.11), we can differentiate the summation with respect to the weights and biases in the network:

$$\mathcal{S}_m^{\mathcal{L}} = \sum_{k=1}^{n} \left(\alpha_k^{\mathcal{L}-1} \cdot \omega_{km}^{\mathcal{L}} \right) + \beta_m^{\mathcal{L}} \qquad \rightarrow \frac{\delta}{\delta \omega_{nm}^{\mathcal{L}}} \sum_{k=1}^{n} \left(\alpha_k^{\mathcal{L}-1} \cdot \omega_{km}^{\mathcal{L}} \right) + \beta_m^{\mathcal{L}} \qquad = \alpha_n^{\mathcal{L}-1} \tag{2.6.3}$$

$$\mathcal{S}_m^{\mathcal{L}} = \sum_{k=1}^{n} \left(\alpha_k^{\mathcal{L}-1} \cdot \omega_{km}^{\mathcal{L}} \right) + \beta_m^{\mathcal{L}} \qquad \rightarrow \frac{\delta}{\delta \beta_m^{\mathcal{L}}} \sum_{k=1}^{n} \left(\alpha_k^{\mathcal{L}-1} \cdot \omega_{km}^{\mathcal{L}} \right) + \beta_m^{\mathcal{L}} \qquad = 1 \tag{2.6.4}$$

Substituting into (2.6.2):

$$\frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \alpha_n^{\mathcal{L}-1} \cdot \frac{\delta C}{\delta \mathcal{S}_m^{\mathcal{L}}} \qquad \rightarrow \quad \alpha_n^{\mathcal{L}-1} \cdot \frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} \cdot \frac{\delta \alpha_m^{\mathcal{L}}}{\delta \mathcal{S}_m^{\mathcal{L}}} \tag{2.6.5}$$

$$\frac{\delta C}{\delta \beta_m^{\mathcal{L}}} = 1 \cdot \frac{\delta C}{\delta \mathcal{S}_m^{\mathcal{L}}} \qquad \rightarrow \quad 1 \cdot \frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} \cdot \frac{\delta \alpha_m^{\mathcal{L}}}{\delta \mathcal{S}_m^{\mathcal{L}}} \tag{2.6.6}$$

Then it follows with equation (2.5.10):

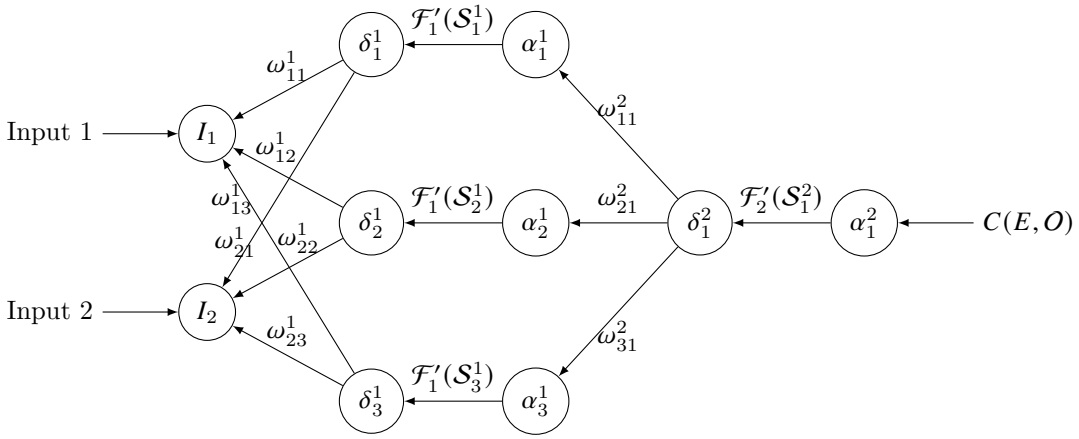
$$\frac{\delta \alpha_m^{\mathcal{L}}}{\delta \mathcal{S}_m^{\mathcal{L}}} = \mathcal{F}'_{\mathcal{L}}(\mathcal{S}_m^{\mathcal{L}}) \qquad \rightarrow \frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \alpha_n^{\mathcal{L}-1} \cdot \mathcal{F}'_{\mathcal{L}}(\mathcal{S}_m^{\mathcal{L}}) \cdot \frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} \tag{2.6.7}$$

$$\rightarrow \frac{\delta C}{\delta \beta_m^{\mathcal{L}}} = \mathcal{F}'_{\mathcal{L}}(\mathcal{S}_m^{\mathcal{L}}) \cdot \frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} \tag{2.6.8}$$

As we are moving backwards through the network, we use the same theory that the gradient at the node is the sum of the gradients of the weights it is connected to, Where are influenced by the gradient at the node it is connected to. We call this value δ.

$$\frac{\delta C}{\delta \alpha_m^{\mathcal{L}}} = \sum_{p=1}^{q} \left(\frac{\delta C}{\delta \omega_{mp}^{\mathcal{L}+1}} \right) = \sum_{p=1}^{q} \left(\delta_p^{\mathcal{L}+1} \cdot \omega_{mp}^{\mathcal{L}+1} \right) \tag{2.6.9}$$

Referencing to our example network:



$I = 2, m = 3, q = 1, l = 1$

This sum is the result of propagating error back through the network and is the backbone of the back propagation algorithm. Finally substituting this back into (2.6.7) and (2.6.8) we get the following:

$$\frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \alpha_n^{\mathcal{L}-1} \cdot \mathcal{F}_{\mathcal{L}}'(\mathcal{S}_m^{\mathcal{L}}) \cdot \sum_{p=1}^{q} \left(\delta_p^{\mathcal{L}+1} \cdot \omega_{mp}^{\mathcal{L}+1} \right) \tag{2.6.10}$$

$$\frac{\delta C}{\delta \beta_m^{\mathcal{L}}} = \mathcal{F}_{\mathcal{L}}'(\mathcal{S}_m^{\mathcal{L}}) \cdot \sum_{p=1}^{q} \left(\delta_p^{\mathcal{L}+1} \cdot \omega_{mp}^{\mathcal{L}+1} \right) \tag{2.6.11}$$

Using what we derived, we can find a formula for the delta values of the nodes in the network, collecting like terms from the equations above(2.6.10), (2.6.11).

$$\frac{\delta C}{\delta \omega_{nm}^{\mathcal{L}}} = \alpha_n^{\mathcal{L}-1} \cdot \delta_m^{\mathcal{L}} \tag{2.6.12}$$

$$\frac{\delta C}{\delta \beta_m^{\mathcal{L}}} = \delta_m^{\mathcal{L}} \tag{2.6.13}$$

These delta values can be used in the form of matrices in our network to update the weights and biases. For the output layer, the delta value is the output of the cost function derivative, as 100% of the error comes through the output layer, and then we apply the derivative of the activation function.

$$\begin{gathered} \delta_m^{\mathcal{L}} = C(E_m, O_m) \cdot \mathcal{F}_m'(\mathcal{S}_m^{\mathcal{L}}) \\ \text{Matrix } \delta_m^{\mathcal{L}} \to [1 \times m] : \delta_m^{\mathcal{L}} = \left| C(E_1, O_1) \cdot \mathcal{F}_1'(\mathcal{S}_1^{\mathcal{L}}), \quad \cdots, \quad C(E_m, O_m) \cdot \mathcal{F}_m'(\mathcal{S}_m^{\mathcal{L}}) \right| \end{gathered} \tag{2.6.14}$$

Where $C(C_m, O_m)$ is the cost function derivative, E_m is the expected output and O_m is the predicted output of the node.

We can then construct an equation for the delta values of the hidden nodes in the network.

For the hidden nodes in layers $1 \leq k < \mathcal{L}$*:*

$$\begin{gathered} \delta_n^k = \mathcal{F}_k'(\mathcal{S}_n^k) \cdot \sum_{p=1}^{q} \left(\delta_p^{k+1} \cdot \omega_{np}^{k+1} \right) \\ \sum_{p=1}^{q} \left(\delta_p^{k+1} \cdot \omega_{np}^{k+1} \right) \to \text{ Matrix } [1 \times p] \cdot [n \times p]^T \to [1 \times n] \\ \text{Matrix } \delta_n^k \to [1 \times n] \circ [1 \times n] \to [1 \times n] : \delta_n^k = \left| \mathcal{F}_k'(\mathcal{S}_1^k) \cdot \textstyle\sum_{p=1}^{q} \left(\delta_p^{k+1} \cdot \omega_{1p}^{k+1} \right), \cdots, \mathcal{F}_k'(\mathcal{S}_n^k) \cdot \sum_{p=1}^{q} \left(\delta_p^{k+1} \cdot \omega_{np}^{k+1} \right) \right| \end{gathered} \tag{2.6.15}$$

With n nodes in layer k, p nodes in layer $k+1$ and ω_{nq}^{k+1} being the weight between node n in layer k and node q in layer $k+1$. With $\circ$ being the element-wise multiplication of the two matrices.

Using our findings from equation (2.6.10) and (2.6.11), we can update the weights and biases in the network. We can update the weights and biases in the network by adding the gradient, multiplied by the learning rate η. This decides how much of the gradient we want to apply to the weights and biases for our next forward pass.

$$\text{Matrix } \omega_{nq}^{k} \rightarrow [n \times q] + [1 \times n]^T \cdot [1 \times q] \rightarrow [n \times q] : \omega_{nq}'^{k} = \omega_{nq}^{k} + \eta \cdot \alpha_{n}^{k-1} \cdot \delta_{q}^{k} \tag{2.6.16}$$

$$\text{Matrix } \beta_{nq}'^{k} \rightarrow [n \times q] + [1 \times q] \rightarrow [n \times 1] : \beta_{nq}'^{k} = \beta_{nq}^{k} + \eta \cdot \delta_{q}^{k} \tag{2.6.17}$$

It is worth remembering that we only update the weights and biases after we have calculated all the delta values for the network, as the error of the network is dependent on what these values were when the forward pass was made. Once we calculate the delta values for the network, we can then update the weights and biases.

2.6.1 Implementation of a single hidden layer network

A single hidden layer can be implemented with a defined structure. As we know how many nodes and layers there are, we can hard code the dimensions. We can initialise the weights and biases randomly, with the structure that we define.

back_prop_single.py

```
import numpy as np

# Define the structure of the network
perceptronStructure = (7, 16, 1)

def init_structure(perceptronStructure):
    '''Intialises the perceptron structure with random weights'''

    # Weights and biases for input layer to the hidden layer
    hiddenWeightMatrix = np.random.uniform(-1, 1, (perceptronStructure[0], perceptronStructure[1]))
    hiddenBiasMatrix = np.random.uniform(-1, 1, (1, perceptronStructure[1]))

    # Weights and biases for hidden layer to the output layer
    outputWeightMatrix = np.random.uniform(-1, 1, (perceptronStructure[1], perceptronStructure[2]))
    outputBiasMatrix = np.random.uniform(-1, 1, (1, perceptronStructure[2]))

    # Create lists for input and output
    hiddenList = [hiddenWeightMatrix, hiddenBiasMatrix]
    outputList = [outputWeightMatrix, outputBiasMatrix]

    return hiddenList, outputList


hiddenList, outputList = init_structure(perceptronStructure)
```

In this example, we have a network with 7 input nodes, 16 hidden nodes and 1 output node.

We iterate through each of the data points and pass them through the forward pass of the network. We pass them through a scaling function to normalise the data, then pass them through the network. By doing this, the network can appropriately learn to predict the output as all datapoints are are passed through an activation function that outputs between 0 and 1.

back_prop_single.py

```
# Normalization function
def min_max_scaler(data):
    '''Function to normalize the data'''
    return (data - data.min()) / (data.max() - data.min())

def min_max_reverser(data, dataMax, dataMin):
    '''Function to normalize the data'''
    return data * (dataMax - dataMin) + dataMin


def forward_pass(data, hiddenList, outputList):
    '''Forward pass of the neural network'''

    # Calculate for the hidden layer
    hiddenSum = (inputData @ hiddenList[0]) + hiddenList[1]
    hiddenActivated = sigmoid(hiddenSum)

    # Calculate for the output layer
    outputSum = (hiddenActivated @ outputList[0]) + outputList[1]
    outputActivated = sigmoid(outputSum)

```

```
    return hiddenSum, hiddenActivated, outputSum, outputActivated


while count < epochs:

    # Iterate through the training data
    for i, day in enumerate(trainingData.itertuples(index=False), start=0):

        inputData = np.array([float(day[2]), float(day[3]), float(day[4]), float(day[5]), float(day[6]), float(day[7]), float(day[8])])
        expectedOutput = np.array([float(day[1])])

        # Forward pass
        hiddenSum, hiddenActivated, outputSum, outputActivated = forward_pass(inputData, hiddenList, outputList)
```

We can then expand by introducing the backward pass to update the weights and biases of the network.

```
back_prop_single.py

def backward_pass(expectedOutput, hiddenSum, hiddenActivated, outputSum, outputActivated, hiddenList, outputList, inputData, dfLength):
    '''Backward pass of the neural network'''

    # Calculate the cost of the perceptron structure
    structureCost = cost_function_derivative(expectedOutput, outputActivated, dfLength)

    # Calculate the delta values for the output layer
    outputDelta = structureCost * sigmoid_derivative(outputSum)

    # Calculate the delta values for the hidden layer
    hiddenDelta = (outputDelta @ outputList[0].T) * sigmoid_derivative(hiddenSum)


    # Update the weights and biases for the output layer
    outputList[0] += (hiddenActivated.T @ outputDelta) * learningRate
    outputList[1] += outputDelta * learningRate

    # Update the weights and biases for the hidden layer
    hiddenList[0] += (inputData.T @ hiddenDelta) * learningRate
    hiddenList[1] += hiddenDelta * learningRate

    return (hiddenList, outputList)


while count < epochs:

    # Iterate through the training data
    for i, day in enumerate(trainingData.itertuples(index=False), start=0):

        inputData = np.array([float(day[2]), float(day[3]), float(day[4]), float(day[5]), float(day[6]), float(day[7]), float(day[8])])
        expectedOutput = np.array([float(day[1])])

        # Forward pass
        hiddenSum, hiddenActivated, outputSum, outputActivated = forward_pass(inputData, hiddenList, outputList)
        hiddenList, outputList = backward_pass(expectedOutput, hiddenSum, hiddenActivated, outputSum, outputActivated, hiddenList, outputList, inputData.reshape(1, -1), dfLength)

    count += 1
```

We can then create predictions after a given amount of epochs, and calculate the error of the network. We can then plot these predictions against the expected output to see how well the network is performing.

2.6.2 Multiple Hidden Layers

The first improvement to the network is to add more hidden layers. This allows the network to learn more complex patterns in the data, as the network can learn more complex functions. We can iteratively generate hidden layers and then use the same process to calculate changes and weight updates.

```
back_prop_multi.py

import numpy as np

# Define the structure of the network
perceptronStructure = (7, [(16, 'sigmoid'),(16, 'sigmoid'), (1, 'sigmoid')])

# Initialise Perceptron Structure
```

```
def init_structure(perceptronStructure):
    '''Initialises the perceptron structure'''

    # Split structure
    inputDimensions = perceptronStructure[0]
    nodeDimensions = perceptronStructure[1]

    # Create lists for weights and biases
    weightList = []
    biasList = []

    # Initialise the first layer of the perceptron
    weightMatrix = np.random.uniform(-1, 1, (inputDimensions, nodeDimensions[0][0]))
    biasMatrix = np.random.uniform(-1, 1, (1, nodeDimensions[0][0]))

    # Append to the lists
    weightList.append(weightMatrix)
    biasList.append(biasMatrix)

    # Initialise the node layers of the perceptron
    for i in range(1, len(nodeDimensions)):
        weightMatrix = np.random.uniform(-1, 1, (nodeDimensions[i - 1][0], nodeDimensions[i][0]))
        biasMatrix = np.random.uniform(-1, 1, (1, nodeDimensions[i][0]))

        # Append to the lists
        weightList.append(weightMatrix)
        biasList.append(biasMatrix)

    return weightList, biasList
```

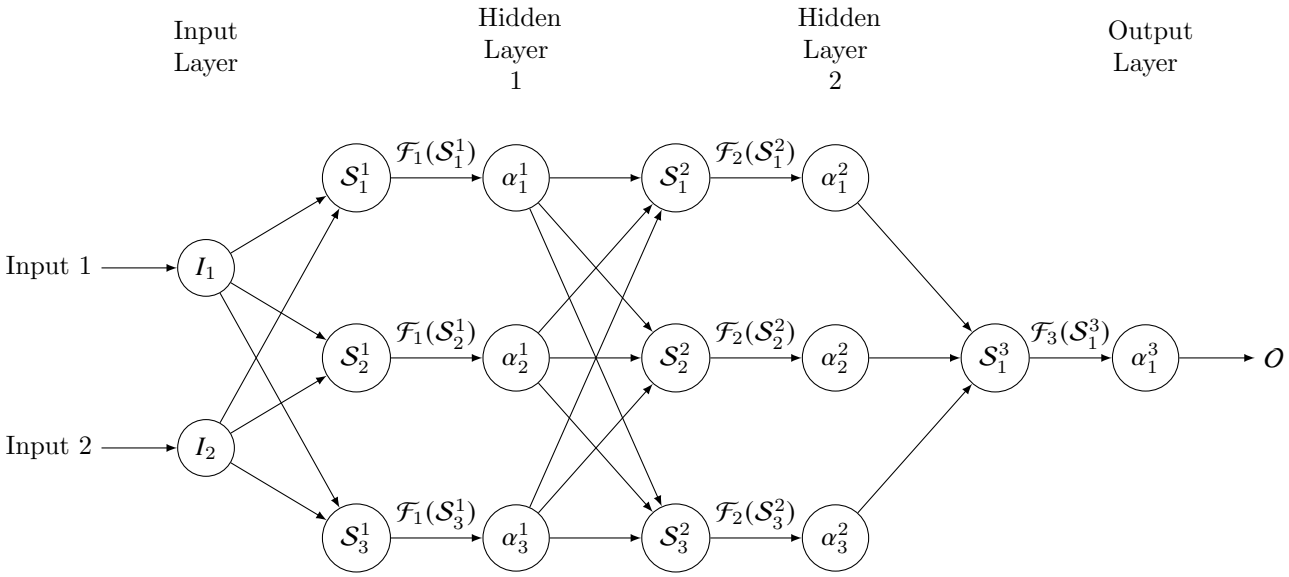
As each layer can have its own activation function, we can define the activation function for each layer. To do this, each layer has a tuple with the number of nodes and the activation function. These are then stored as a list and joined with the input dimensions to create the structure of the network.

The structure we use to iterate through the layers is the same as our backwards pass. We calculate our first item, and then iterate through the rest of the layers until we reach the end.

However, for the forward pass, we just go through the whole list and calculate the forward pass for each layer iteratively. This is because it is simpler and each step is the same computation

back_prop_multi.py

```
import numpy as np

layerStructure = perceptronStructure[1]

def forward_pass(inputData, weightList, biasList, layerStructure):
    '''Forward pass of the neural network'''

    # Create a list for the activated nodes and summed nodes
    summedList = []
    activatedList = []


    # Calculate for the hidden layers
    for layer, structure in enumerate(layerStructure):
        # Get activation function
        activationFunction = get_activation(structure[1])

        # Calculate the sum of the matrix
        sumMatrix = (inputData @ weightList[layer]) + biasList[layer]

        activatedMatrix = activationFunction(sumMatrix)

        # Append to the list
        summedList.append(sumMatrix)
        activatedList.append(activatedMatrix)

        # Update the input data
        inputData = activatedMatrix

    return activatedList, summedList
```


Input Layer
Hidden Layer 1
Hidden Layer 2
Output Layer
Input 1
Input 2
I_1
I_2
$\mathcal{S}_1^1$
$\mathcal{S}_2^1$
$\mathcal{S}_3^1$
$\mathcal{F}_1(\mathcal{S}_1^1)$
$\mathcal{F}_1(\mathcal{S}_2^1)$
$\mathcal{F}_1(\mathcal{S}_3^1)$
α_1^1
α_2^1
α_3^1
$\mathcal{S}_1^2$
$\mathcal{S}_2^2$
$\mathcal{S}_3^2$
$\mathcal{F}_2(\mathcal{S}_1^2)$
$\mathcal{F}_2(\mathcal{S}_2^2)$
$\mathcal{F}_2(\mathcal{S}_3^2)$
α_1^2
α_2^2
α_3^2
$\mathcal{S}_1^3$
$\mathcal{F}_3(\mathcal{S}_1^3)$
α_1^3
O